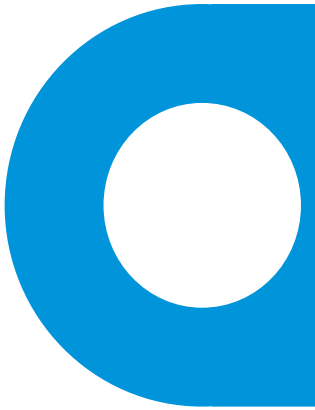


"Ss. Cyril and Methodius" University in Skopje
FACULTY OF COMPUTER
SCIENCE AND ENGINEERING



FACULTY OF COMPUTER SCIENCE AND ENGINEERING

Functional Interfaces, Lambda Expressions

Advanced Programming

FCSE 2026

Nested Classes

- **Nested class:** Any class declared within the body of another class or interface; **inner classes** are strictly the *non-static* nested forms.
- **Design benefits:** Groups helper types logically, hides implementation details via strong encapsulation, and keeps single-use code close to where it operates.
- **Two top-level categories:**
  - **Static nested classes:** Declared as static members of the enclosing class (nested, *not* inner).
  - **Inner classes:** Non-static member inner classes, named local classes, and anonymous classes.
- **Orthogonal to inheritance:** Nesting establishes lexical scope and member access privileges, not an extends (*is-a*) hierarchy.

Classification Summary

Category	Variety	Declared With static?	Considered an "Inner Class"?
Static Nested	Static member class	Yes	No
Inner	Member inner class	No	Yes
Inner	Named local class	No (inside block/method)	Yes
Inner	Anonymous class	No (expression-level)	Yes

Static Nested Classes

- **No enclosing instance:** Behaves like a top-level class placed inside the enclosing class's namespace; instantiated without an Outer object via new Outer.Nested().
- **Direct static access:** Can directly read and modify all static members of the enclosing class, including private ones.
- **Instance access requires a reference:** Cannot directly access enclosing instance fields or methods; must use an explicit Outer object reference.
- **Ideal use case:** Value summaries, builders, or internal nodes tied to the outer type conceptually but independent of any single outer instance.

Nested Class Example

```
public class BankAccount {
    private static double baseFee = 2.50;
    private double balance; // constructor and methods omitted

    public static class FeeSummary {
        private final double finalFee;

        public FeeSummary(BankAccount account) {
            // Direct access to private static field; explicit reference for
instance field
            this.finalFee = (account.balance >= 1000.0) ? 0.0 : baseFee;
        }
        public double getFinalFee() { return finalFee; }
    }
}
// External instantiation (no BankAccount instance needed to reference the type):
// BankAccount.FeeSummary summary = new BankAccount.FeeSummary(existingAccount);
```

Member Inner Classes

- **Bound to an enclosing instance:** Every member inner class instance is implicitly associated with the specific Outer object that created it.
- **Full member access:** Directly accesses all static and instance members of its enclosing object—including private fields—without an explicit reference.
- **Instantiation syntax:** Created inside Outer instance methods via new Inner(), or externally via an existing instance using outerRef.new Inner().
- **Enclosing reference qualification:** Inside the inner class, this refers to the inner instance and Outer.this refers to the enclosing instance.

Inner Class Example

```
public class BankAccount {
    private double balance = 500.0; // constructors and other methods
omitted

    public class Transaction {
        private final double amount;

        public Transaction(double amount) {
            this.amount = amount;
            balance += amount; // Directly mutates enclosing
BankAccount's private field
        }
        public double getAccountBalance() { return
BankAccount.this.balance; }
    }
}
// External instantiation requires an existing BankAccount instance:
// BankAccount acct = new BankAccount();
// BankAccount.Transaction tx = acct.new Transaction(150.0);
```

Named Local Classes

- **Block-scoped declaration:** Declared inside a method, constructor, or initializer block; visible only within that enclosing block.
- **Variable capture rule:** Can access local variables and method parameters of the enclosing block only if they are **final or effectively final** (assigned once and never modified).
- **Enclosing state access:** Accesses enclosing object instance fields freely (fields do *not* need to be effectively final), unless the local class is declared inside a static method.
- When a local class is needed for only a single expression and does not even need a name, Java provides *anonymous classes*.

Local Class Example

```
public class BankAccount {
    private double balance = 500.0; // constructors and other
methods omitted

    public boolean processWithdrawal(final double amount,
double feeRate) {
        double fee = amount * feeRate; // 'fee' is
effectively final (never reassigned)

        class WithdrawalValidator {
            boolean isValid() {
                return (balance - amount - fee) >= 0.0; //
'balance' is an instance field
            }
        }
        return new WithdrawalValidator().isValid();
    }
}
```

Nesting overview

Feature	Static Nested Class	Member Inner Class	Named Local Class
Declaration Location	Inside class body (static)	Inside class body (non-static)	Inside method, constructor, or block
Visibility / Scope	Controlled by access modifiers	Controlled by access modifiers	Restricted to enclosing block
Enclosing Instance Needed?	No (new Outer.Nested())	Yes (outer.new Inner())	Yes in instance methods; No in static methods
Access to Outer Members	static members only (direct)	All static and instance members	Follows enclosing method context + effectively final locals
Typical Use Case	Independent helper, builder, summary record	Helper tightly coupled to one Outer object	Method-specific helper needing a class name or constructor



Anonymous Classes

- An anonymous class in Java is a **local class** declared **without a name.** It allows for the **declaration and instantiation** of a class **simultaneously**, typically **used for one-time implementations of interfaces or extensions of classes** where defining a separate, named class would be unnecessary.
- No name, single use, creates exactly one object
- Derived from other class or interface
- Using the new keyword followed by the interface or class name, immediately followed by curly braces {} containing the class definition, including overridden methods and any new member



Anonymous Class Example

```
class Book {
    private String title;
    Book(String t) {
        title=t;
    }
    public String getDescription() {
        return "";
    }
    @Override
    public String toString() {
        return title + " " + getDescription();
    }
}

public class Main {
    public static void main(String[] args) {
        Book b1 = new Book("noTitle");
        Book bd = new Book("DesignPatterns") {
            @Override
            public String getDescription() { return "Famous GoF book."; }
        };
        System.out.println(b1);
        System.out.println(bd);
    }
}
```

b1 – refers to a standard Book-class object

bd - refers to the single object of an unnamed class derived from Book

```
noTitle
DesignPatterns Famous GoF book.
```

Anonymous Classes

- Anonymous classes allow the code to be more concise.
- They allow you to declare and instantiate a class at the same time.
- They are like local classes, except they have no name. They are used if you only need to use a local-class instance only once.

Instance Initializer

```
static initializer called
instance initializer called
constructor called
instance initializer called
constructor called
```

```
public class Foo {
    //instance variable initializer
①   String s = "abc";
    //constructor
③   public Foo() {
        System.out.println("constructor called");
    }
    //static initializer
⓪   static {
        System.out.println("static initializer called");
    }
②   //instance initializer
    {
        System.out.println("instance initializer called");
    }
    public static void main(String[] args) {
        new Foo();
        new Foo();
    }
}
```

Anonymous Classes

```
public class HelloWorldAnonymousClasses {
    interface HelloWorld {
        void greet();
        void greetSomeone(String someone);
    }
    public void sayHello() {
        class EnglishGreeting implements HelloWorld {
            String name = "world";
            @Override
            public void greet() {
                greetSomeone("world");
            }
            @Override
            public void greetSomeone(String someone) {
                name = someone;
                System.out.println("Hello " + name);
            }
        }
        HelloWorld englishGreeting = new EnglishGreeting();
```

Anonymous Classes

```
HelloWorld frenchGreeting = new HelloWorld() {
    String name = "tout le monde";
    @Override
    public void greet() {
        greetSomeone("tout le monde");
    }
    @Override
    public void greetSomeone(String someone) {
        name = someone;
        System.out.println("Salut " + name);
    }
};
```

Anonymous Classes

```
HelloWorld spanishGreeting = new HelloWorld() {
        String name = "mundo";
        @Override
        public void greet() {
            greetSomeone("mundo");
        }
        @Override
        public void greetSomeone(String someone) {
            name = someone;
            System.out.println("Hola, " + name);
        }
    };
    englishGreeting.greet();
    frenchGreeting.greetSomeone("Fred");
    spanishGreeting.greet();
}
public static void main(String[] args) {
    HelloWorldAnonymousClasses myApp =
            new HelloWorldAnonymousClasses();
    myApp.sayHello();
} }
```

Anonymous Classes

- The syntax for defining (and instantizing) an anonymous class is like calling a constructor, followed by a class definition contained in a block of code.

```
HelloWorld frenchGreeting = new HelloWorld() {
    String name = "tout le monde";
    @Override
    public void greet() {
        greetSomeone("tout le monde");
    }
    @Override
    public void greetSomeone(String someone) {
        name = someone;
        System.out.println("Salut " + name);
    }
};
```

Defining an Anonymous Class

An expression defining an anonymous class consists of:

- The **new** operator
- The name of the **interface** to be implemented or the **class** to be expanded. In this example, the anonymous class implements the HelloWorld interface.
- Parentheses containing **constructor arguments**, just like a common expression for creating an object. Note: When implementing an interface, there is no constructor, so an empty pair of parenthesis is used, as in this example.
- A **body** that is a declaration of class. Specifically, in the body, declarations of methods are allowed, but not expressions.

Defining an Anonymous Class

- Like local classes, anonymous classes can capture variables; they have the same access to local variables of the enclosing scope:
  - An anonymous class has access to the members of its enclosing class.
  - An anonymous class cannot access local variables in its enclosing scope that are not declared as `final` or *effectively final*.
  - Like a nested class, a declaration of a type (such as a variable) in an anonymous class shadows any other declarations in the enclosing scope that have the same name.

```
public class EffectiveFinalDemo{
   public static void main(String args[]) {
      String nonFinal = "I am non final local variable";
      Runnable r = new Runnable() {
         @Override
         public void run() {
            System.out.println("Using non-final local variable inside anonymous class in Java");
            System.out.println("Value of non-final variable is : " + nonFinal);
            // compile time error - must be final or effective final
            // nonFinal = "Can I change non-final variable inside anonymous class";
         }
      };
      Thread t = new Thread(r);
      t.start();
   }
}
```

Runnable in an interface

OK, it is effectively final and only read

Anonymous class restrictions

- Anonymous classes also have the same restrictions as local classes with respect to their members:
  - You cannot declare static initializers or member interfaces in an anonymous class.
  - An anonymous class can have static members provided that they are constant variables.
- Note that you can declare the following in anonymous classes:
  - Fields
  - Extra methods (even if they do not implement any methods of the supertype)
  - Instance initializers
  - Local classes
- However, you cannot declare constructors in an anonymous class.

Functional Interfaces

- Let's create an instance of the Runnable class/interface. We usually do this using anonymous class like in the example below.

```
Runnable r = new Runnable() {
        @Override
        public void run() {
                System.out.println("My Runnable");
        }
};
```

- If you look at the code above, the **actual part that is needed** is the code in the **run() method**. The rest of the code is due to the way Java programs are structured.
- Java 8 functional interfaces and lambda expressions help us write shorter and cleaner code by removing much of the template code.

Functional Interfaces

- An interface with **exactly one abstract method** is called a **functional interface**. **@FunctionalInterface annotation** is added to mark the interface as a *functional interface*.
- Annotation is not mandatory, but it is good practice to avoid accidentally adding additional methods. If an interface is labeled with `@FunctionalInterface` annotation, attempting to have more than one abstract method in it will cause a compiler error.
- The main benefit of Java 8 functional interfaces is that we can use *lambda expressions* to instantiate them and avoid using cumbersome implementations of anonymous classes.
  - Java 8 Collections API has been rewritten and new Stream API is introduced that uses a lot of functional interfaces. Java 8 has defined a lot of functional interfaces in ***java.util.function*** package. Some of the useful java 8 functional interfaces are ***Consumer, Supplier, Function*** and ***Predicate*** (https://docs.oracle.com/javase/8/docs/api/java/util/function/package-summary.html).

Lambda expressions

- Lambda expressions were introduced in Java 8 and are considered one of the greatest features of Java 8. Lambda expressions facilitate functional programming and greatly simplify development.
- The lambda expression is characterized by the following syntax

```
parameter -> expression body
```

Important characteristics of lambda expression

- **Optional type declaration** – There is no need to declare the type of parameter. The compiler can infer the same from the parameter value.
- **Optional parenthesis around parameter** – There is no need to declare a single parameter in parentheses. For more parameters, parentheses are needed.
- **Optional curly braces** – There is no need to use large parentheses in the body of an expression if it contains only one statement.
- **Optional return keyword** – The compiler automatically returns the value of the expression in the case of a single expression. Curly parentheses are necessary to indicate that the expression returns value.

Reminder

```
@FunctionalInterface
public interface Articulate {
    void makeSound();
}
abstract class Animal {
    protected String name;
    public Animal(String n) { name=n; }
}

class Dog extends Animal implements Articulate {
    Dog(String n) { super(n); }
    @Override
    public void makeSound() {
        System.out.println(name+ ": Bark bark");
    }
}
class Cat extends Animal implements Articulate {
    Cat(String n) { super(n); }
    @Override
    public void makeSound() {
        System.out.println(name+ ": Meow meow");
    }
}
```

Reminder

```
public class Main {
    public static void main(String[] args) {
            Articulate [] a = {
                    new Cat("Pussy"),
                    new Dog("Puppy")
            };

        for(Articulate x : a) {
            //x.makeSound();
            communicate(x);
        }
    }
    static void communicate(Articulate a) {
        a.makeSound();
    }
}
```

```
Pussy: Meow meow
Puppy: Bark bark
```

Reminder

```
public class Human implements Articulate {
    protected String name, surname;
    public Human(String n, String s) {
        name=n;
        surname = s;
    }
    @Override
    public void makeSound() {
        System.out.println(name + " " + surname
                + ": I can speak");
    }
}
```

Reminder

```
public class Main {
    public static void main(String[] args) {
            Articulate [] a = { new Cat("Pussy"),
                    new Dog("Puppy"),
                    new Human("John", "Smith")
            };

        for(Articulate x : a) {
            //x.makeSound();
            communicate(x);
        }
    }
    static void communicate(Articulate a) {
        a.makeSound();
    }
}
```

```
Pussy: Meow meow
Puppy: Bark bark
John Smith: I can speak
```

The only thing we can do with this object in the body of the communicate method is to call the only method (makeSound) declared in the interface. The behavior of communicate will only differ depending on the implementation of makeSound for the argument a.

Lambda

- Let's say we need a specific **`makeSound`** implementation to be called up by **`communicate`**
- To do this, you need to:
  - Create a NEW **`XYZ`** class that will implement the **`Articulate`** interface
  - Write an implementation of **`makeSound`** method in **`XYZ`**
  - Create an object of the **`XYZ`** class
  - Call **`communicate`** with this object as a parameter
  - ... that will call our implementation of **`makeSound`**

Lambda

- Let's say we need a specific **`makeSound`** implementation to be called up by **`communicate`**
- To do this, you need to:
  - Create a NEW **`XYZ`** class that will implement the **`Articulate`** interface
  - Write an implementation of **`makeSound`** method in **`XYZ`**
  - Create an object of the **`XYZ`** class
  - Call **`communicate`** with this object as a parameter
  - ... that will call our implementation of **`makeSound`**

No Lambda implementation

```
public class Robot implements Articulate {
    @Override
    public void makeSound() {
        System.out.println("R2D2 : 1010001101");
    }
}

...

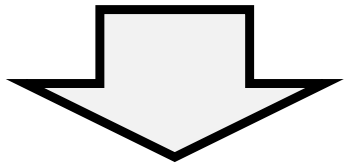
Robot r = new Robot();
communicate(r);
```

```
R2D2 : 1010001101
```

Lambda

Can we somehow get the job done by just writing makeSound without writing an entire class?

```
communicate(
                        () -> {
    System.out.println("R2D2 : 1010001101");
}
);
```



```
communicate( () -> System.out.println("R2D2 : 1010001101") );
```

Lambda

Can we somehow get the job done by just writing makeSound without writing an entire class?

```
communicate(
        void makeSound()    {
    System.out.println("R2D2 : 1010001101");
}
);
```

`communicate( )` accepts as argument an object that implements the `Articulate` interface. `Articulate` is a *functional interface*, meaning that it defines **exactly one** method. The name of the method `makeSound` is redundant.

Lambda

Can we somehow get the job done by just writing makeSound without writing an entire class?

```
communicate(
        void makeSound()    {
    System.out.println("R2D2 : 1010001101");
}
);
```

In the `Articulate` interface the definition of the method defines what result it returns and how many arguments it accepts. `void` is redundant.

Lambda

Can we somehow get the job done by just writing makeSound without writing an entire class?

```
communicate(
        void makeSound()    {
    System.out.println("R2D2 : 1010001101");
}
);
```

If we add **->** between ( ) and the body of the function { ... } we will get a valid **lambda expression**.

Lambda

Can we somehow get the job done by just writing makeSound without writing an entire class?

```
communicate(
                        () -> {
    System.out.println("R2D2 : 1010001101");
}
);
```

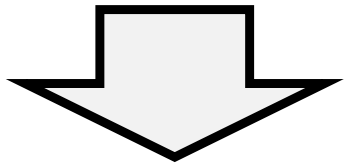
This compiles and works

If the body of the function has only one statement, even the curly braces may be omitted.

Lambda

Can we somehow get the job done by just writing makeSound without writing an entire class?

```
communicate(
                        () -> {
    System.out.println("R2D2 : 1010001101");
}
);
```



```
communicate( () -> System.out.println("R2D2 : 1010001101") );
```

The whole **lambda expression behaves as an object** that implements a functional interface and **can be assigned to a variable**.

Lambda

```
Articulate lambdaC3P0 = () -> System.out.println(
        "C3P0 : I am fluent in over six million forms of communication");
Articulate lambdaGroot = () -> System.out.println("Groot : I am Groot");

communicate(lambdaC3P0);
communicate(lambdaGroot);
```

```
C3P0 : I am fluent in over six million forms of communication
Groot : I am Groot
```

It enables an implementation of a method (**code**) **to be passed** to a method **as a parameter**.

@FunctionalInterface

```
@FunctionalInterface
public interface Calculator {
    int increase(int a, int b);
}
```

Interfaces that have **only one abstract method** are called ***functional interface***. (SAM interface = Single Abstract Method Interface).
Lambda expressions may only be used in the context of a functional interface.

Example

```
public interface Calculator {
    int increase(int a, int b);

}
public class Human implements Calculator {
    protected String name, surname;
    public Human(String n, String s) { name=n; surname = s; }
    @Override
    public int increase(int a, int b)    {
        int result = a+b;
        System.out.println(name + " " + surname + " : "
                + a + " + " + b + " = " + result);
        return result;
    }
}
public class Robot implements Calculator {
    @Override
    public int increase(int a, int b) {
        int result = a+(a*b/100);
        System.out.println("R2D2 : " + a + " + " + b + "% = " + result);
        return result;
   }
}
```

```
John Smith : 1000 + 4 = 1004
John Smith : 2000 + 5 = 2005
John Smith : 3000 + 6 = 3006
SUM=6015
R2D2 : 1000 + 4% = 1040
R2D2 : 2000 + 5% = 2100
R2D2 : 3000 + 6% = 3180
SUM=6320
12000
5985
```

```
public class Main {
    public static void main(String[] args) {
        Calculator[] a = {
                new Human("John", "Smith"),
                new Robot()
        };

        for (Calculator x : a) {
            System.out.println("SUM="+manipulate(x));
        }
        Calculator calcDbl = (x,y) -> { return 2*x; };
        // Calculator calcDbl = (x, y) -> 2*x;
        System.out.println(manipulate(calcDbl));
        System.out.println(manipulate( (i, j) -> i-j ));
    }
    static int manipulate(Calculator a) {
        int accum = 0;
        int[] arr1 = new int[] { 1000, 2000, 3000 };
        int[] arr2 = new int[] { 4, 5, 6 };
        for(int i = 0; i<arr1.length; i++)
            accum += a.increase(arr1[i],arr2[i]);
        return accum;
    }
}
```

```
public class Java8Tester {
   interface MathOperation {
      int operation(int a, int b);
   }

   interface GreetingService {
      void sayMessage(String message);
   }

   private int operate(int a, int b, MathOperation mathOperation){
      return mathOperation.operation(a, b);
   }
```

```
public static void main(String args[]){
    Java8Tester tester = new Java8Tester();
    //with type declaration
    MathOperation addition = (int a, int b) -> a + b;
    //with out type declaration
    MathOperation subtraction = (a, b) -> a - b;
    //with return statement along with curly braces
    MathOperation multiplication = (int a, int b) -> { return a * b; };
    //without return statement and without curly braces
    MathOperation division = (int a, int b) -> a / b;
    System.out.println("10 + 5 = " + tester.operate(10, 5, addition));
    System.out.println("10 - 5 = " + tester.operate(10, 5, subtraction));
    System.out.println("10 x 5 = " + tester.operate(10, 5, multiplication));
    System.out.println("10 / 5 = " + tester.operate(10, 5, division));
    //with parenthesis
    GreetingService greetService1 = message -> System.out.println("Hello " + message);
    //without parenthesis
    GreetingService greetService2 = (message) -> System.out.println("Hello " + message);
    greetService1.sayMessage("Mahesh");
    greetService2.sayMessage("Suresh");
  }
}
```

- Lambda expressions are used primarily to define inline implementation of a functional interface, i.e., an interface with a single method only. In the above example, we've used various types of lambda expressions to define the operation method of MathOperation interface. Then we have defined the implementation of sayMessage of GreetingService.
- Lambda expressions eliminates the need of anonymous class and gives a very simple yet powerful *functional programming* capability to Java.


Questions?